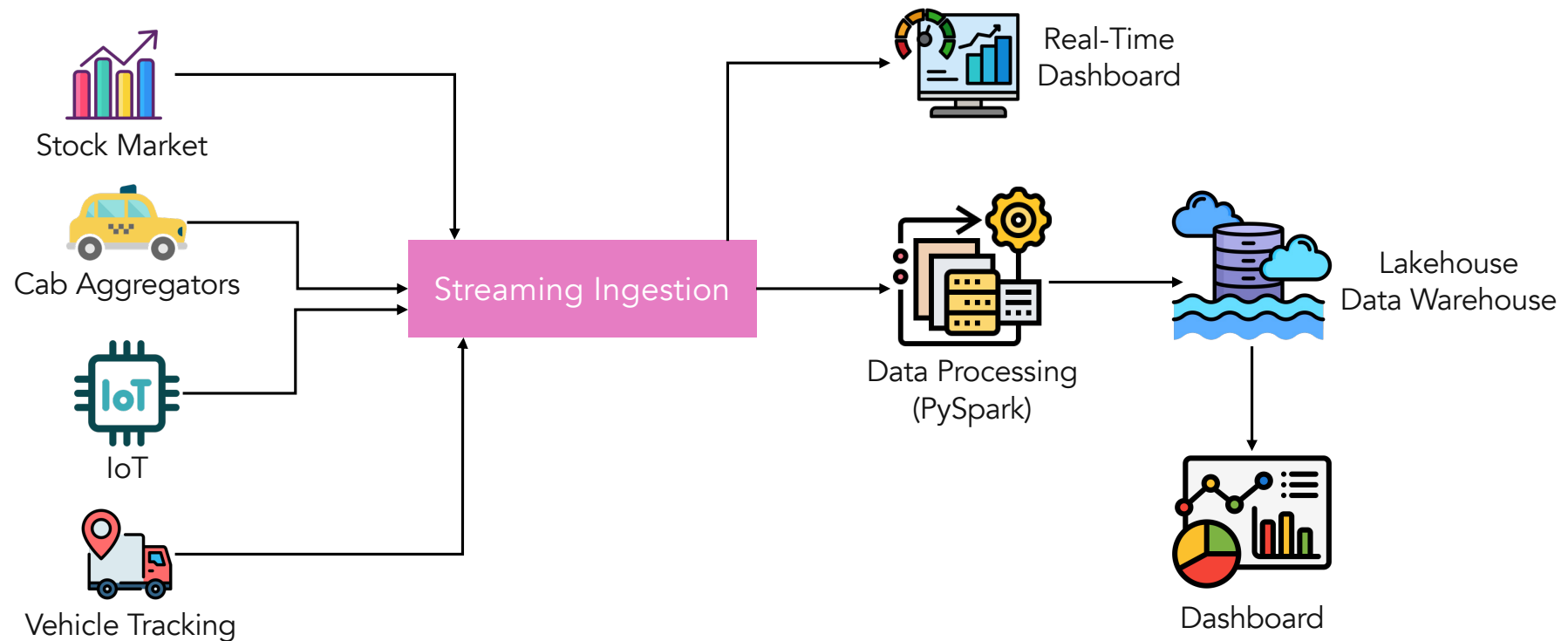


Fabric Real-Time Intelligence

www.3aayaam.in

Real-Time Streaming

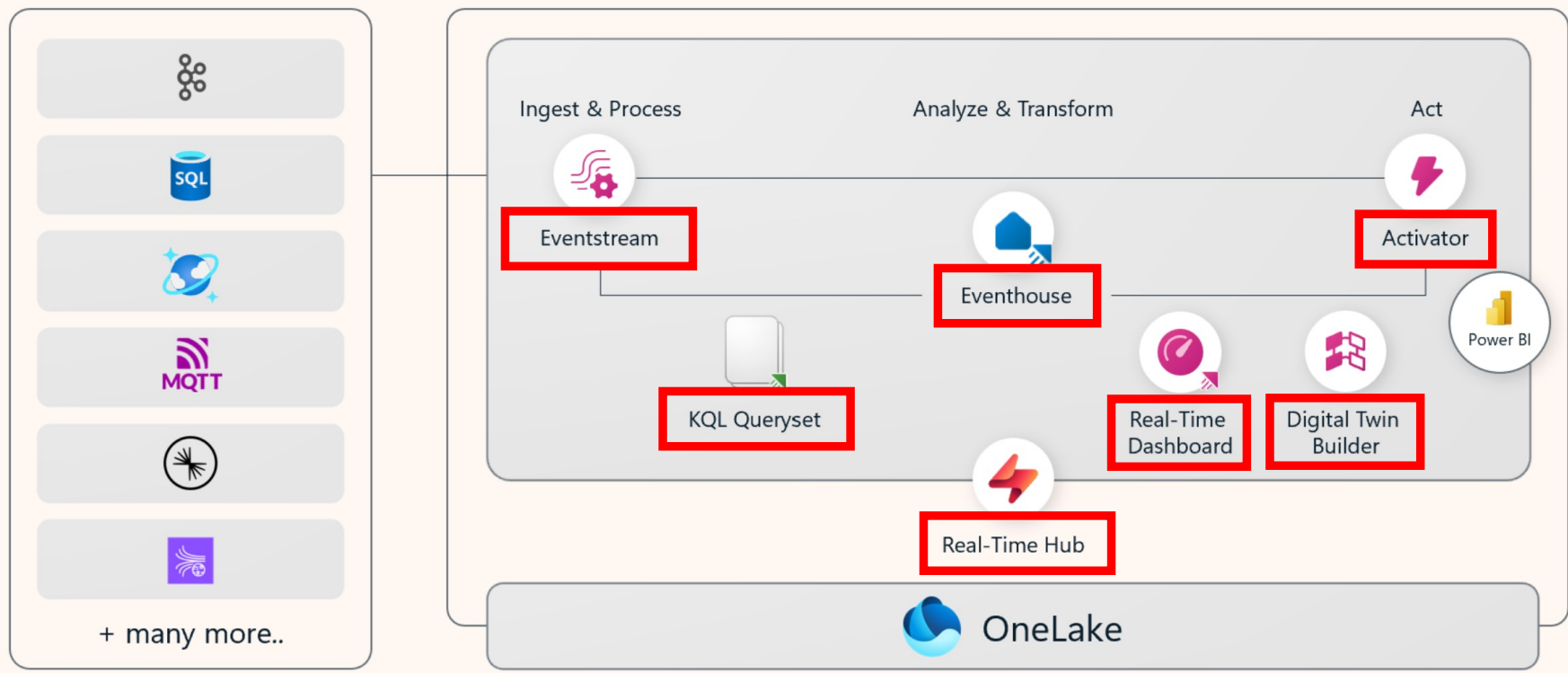
- Real-time Application – Difference in time between data generation & visual is very less.



Real-Time Intelligence (RTI)



Real-Time Intelligence handles data ingestion, transformation, storage, modelling, analytics, visualization, tracking, AI and real-time actions.



RTI Solutions

Capability	Real-time Intelligence solution
Multicloud connectors	Native integration for Confluent Kafka, Amazon Kinesis, and Google Pub/Sub.
Support for CDC streams	Native integration for Azure Cosmos DB, PostgreSQL, MySQL DB, and Azure SQL.
Support for protocols	Azure Event Hubs, AMQP, and Kafka.

Ingest

Capability	Real-time Intelligence solution
Data profiling	The data profiling view of your real-time tables shows out-of-the-box histograms and min-max ranges for each column.
Digital twin modeling	Digital twin builder (preview)
Visual data exploration	Drag features to visually analyze your real-time data.
Copilot experience	Natively available
Built-in ML models	Anomaly detection and forecasting models are available. Business users can also apply anomaly detection models to incoming streaming data.
Visualization (Microsoft)	Native one-click integration with Power BI and real-time dashboard
Visualization (Third party)	Grafana, Kibana, and Matlab can also be integrated with Eventhouse.

Analyze

RTI Solutions

Capability	Real-time Intelligence solution
Driving business actions from insights	Available in Fabric using Fabric Activator items with built-in integration with Power BI semantic models, Eventstream, and KQL queries using KQL Querysets or Real-Time Dashboards.
Notification destinations	Microsoft Teams, Microsoft Outlook, and Power Automate connectors.

Report

Capability	Real-time Intelligence solution
Unified catalog of data streams	Real-time hub: - Data streams created by the users - Existing streams from Microsoft sources - Fabric system eventstreams
Discovery of Microsoft data streams	Real-time Intelligence hub finds data streams in your Azure tenant.
Capture and act on events from Azure Storage	Azure Blob Storage event-based triggers are available.
Capture and act on events from Fabric	Native in Fabric

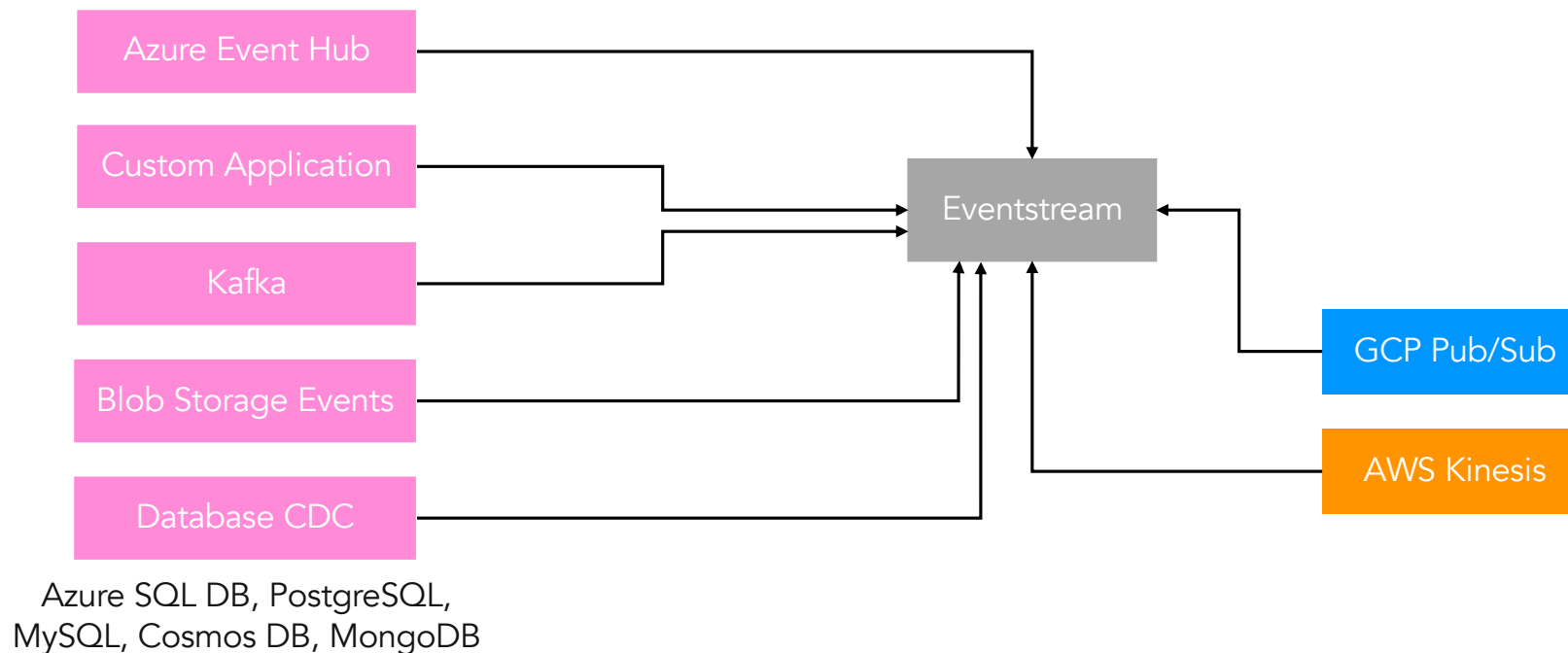
Unified Catalog

RTI Eventstream

www.3aayaam.in

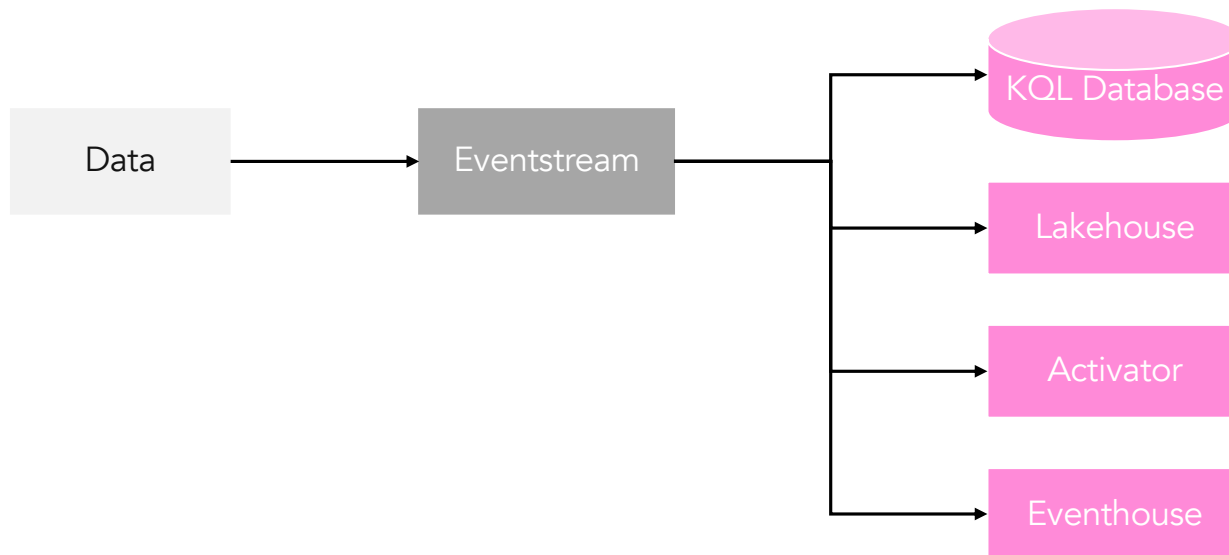
Eventstream - Sources

- Eventstream – Ingests real-time streaming data, event data.
- Multiple Azure and non-Azure data sources are supported.



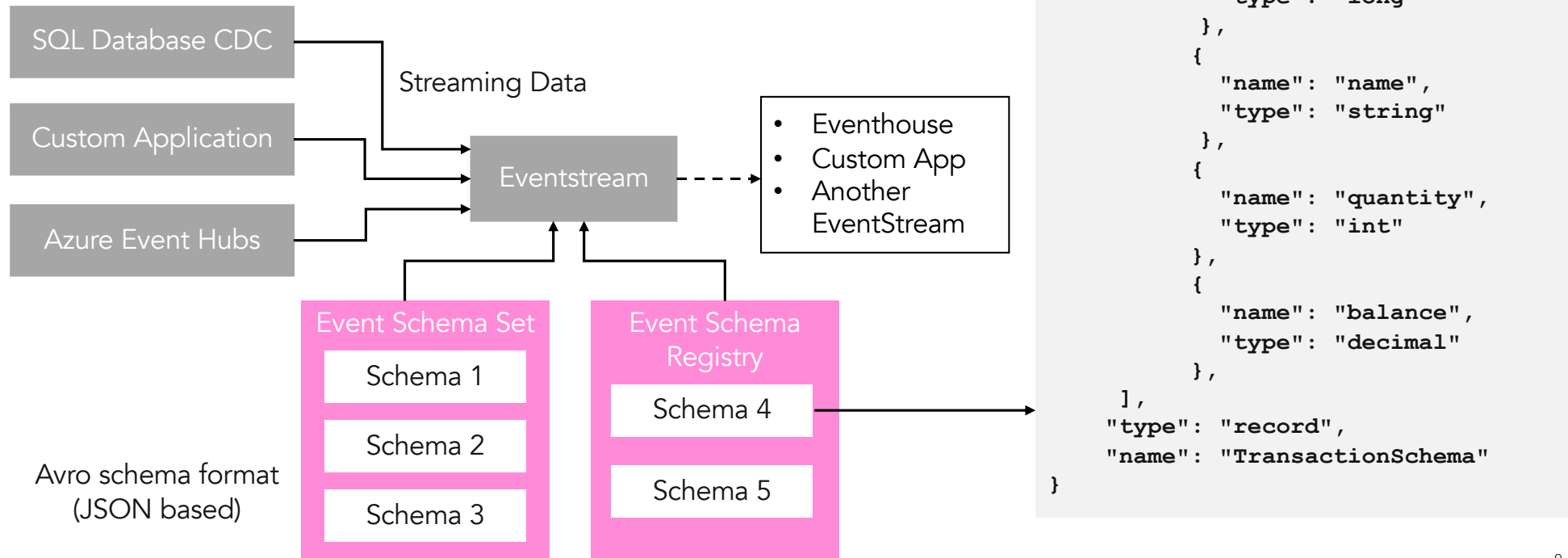
Eventstream - Destinations

- Eventstream provides Apache Kafka endpoint.
 - Data can be sent or read using Kafka endpoint and protocol.
- Data can be routed to different destinations automatically.



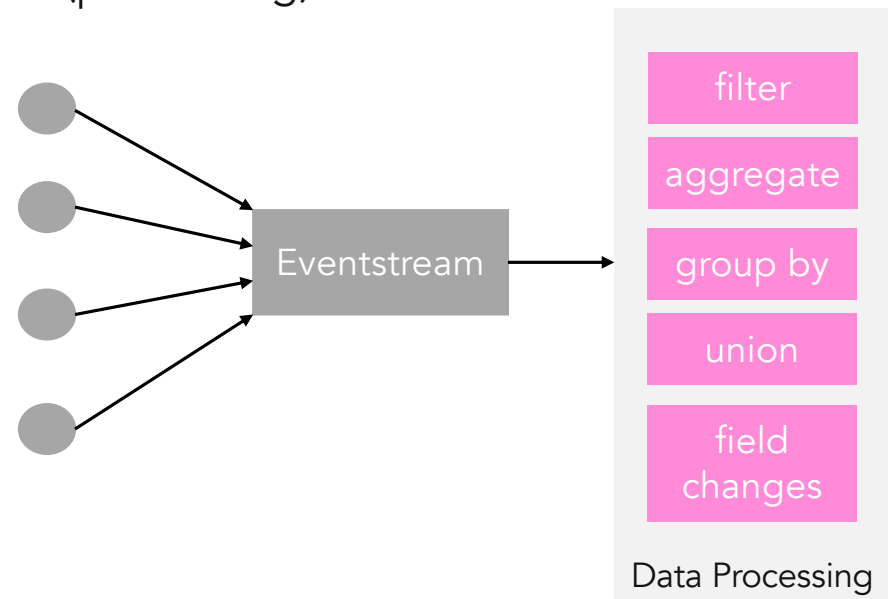
Eventstream - Schema

- Event Schema Set.
- Schema definition for streaming data.



Eventstream – Ingest & Process

- Ingest real-time data into Fabric Data Platform.
- No-code data transformations & aggregations (processing).
 - filter.
 - sum, min, max, avg.
 - group by.
 - union – Union data from 2 streams.
 - join – Join data from 2 streams.
 - add, remove fields/columns.
 - rename fields.
 - Data type modification of fields/columns.



Eventstream – CU Consumption

Operation in Capacity Metrics app	Description	Operation unit of measure	Fabric consumption rate
Eventstream Per Hour ¹	Flat charge	Per hour	0.222 capacity unit (CU) hours
Eventstream Data Traffic Per GB	Data ingress and egress volume in default and derived streams (includes 24-hour retention)	Per gigabyte	0.342 CU hours
Eventstream Processor Per Hour	Computing resources that the processor consumes	Per hour	Starts at 0.778 CU hours and autoscales ² per throughput
Eventstream Connectors Per vCore Hour	Computing resources that the connectors consume	Per hour	0.611 CU hours per vCore ³

Python!!

Visit www.3aayaam.in

[Courses](#)[Blog](#)[About](#)[Contact](#)[FAQ](#)[Login](#)

Python Essentials for Data Engineering



Created by Soumyadeep Dey

 English



Python Essentials for Data Engineering

Buy now for FREE

Language: English

Instructors: Soumyadeep Dey

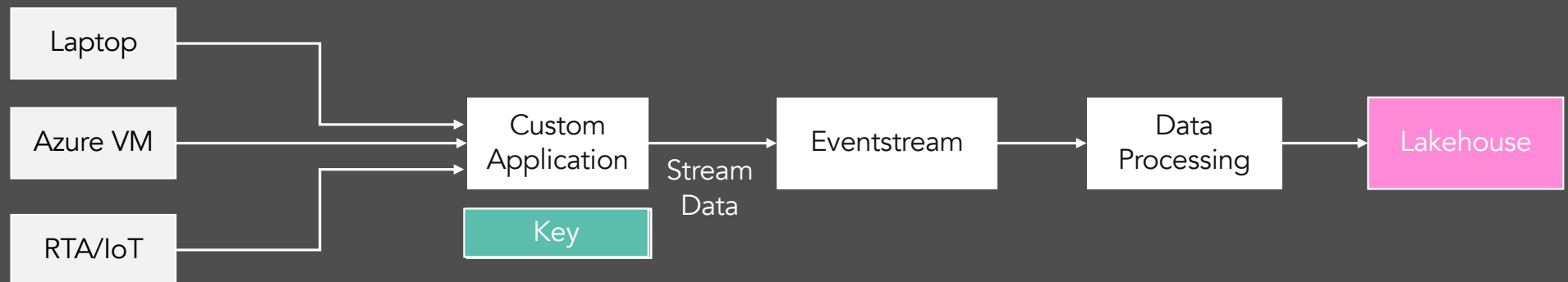
About the course

In this course you will get to learn and work on all the Python concepts and hands on that are needed for a Data Engineer.

www.3aayaam.in

Hands-On 1 : WITHOUT Schema

- Lakehouse, customer application are supported as destinations only using without schema.



- Create Eventstream.
- Add "Custom Endpoint". Do not activate "Schema association".
- Add Lakehouse destination.
- Add "Transformation".
- PUBLISH the Eventstream.

Hands-On 1 : Python Code EventHub

- Python code to send data to Eventstream for Lakehouse using azure-eventhub.

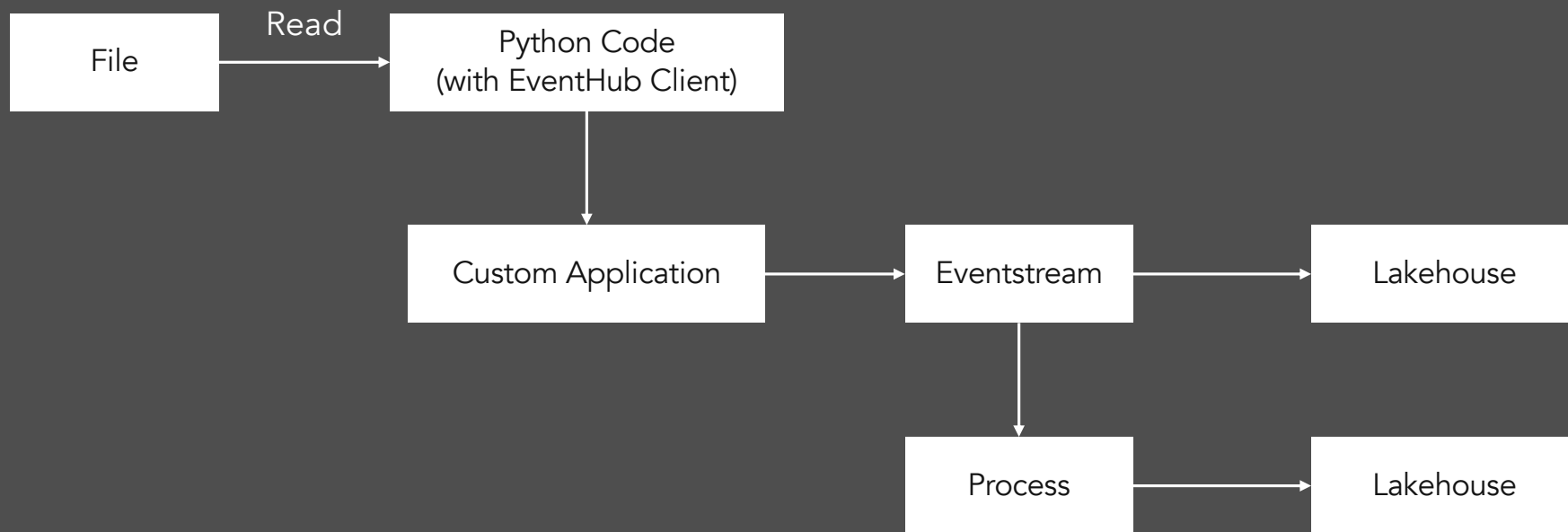
```
import asyncio
from azure.eventhub import EventData
from azure.eventhub.aio import EventHubProducerClient

async def send_data_to_eventstream():
    producer_client = EventHubProducerClient.from_connection_string(
        conn_str=<conn_str>,
        eventhub_name=<eventhub_name>
    )

    async with producer_client:
        event_data_batch = await producer_client.create_batch()
        event_data_batch.add(EventData(event_data))
        await producer_client.send_batch(event_data_batch)

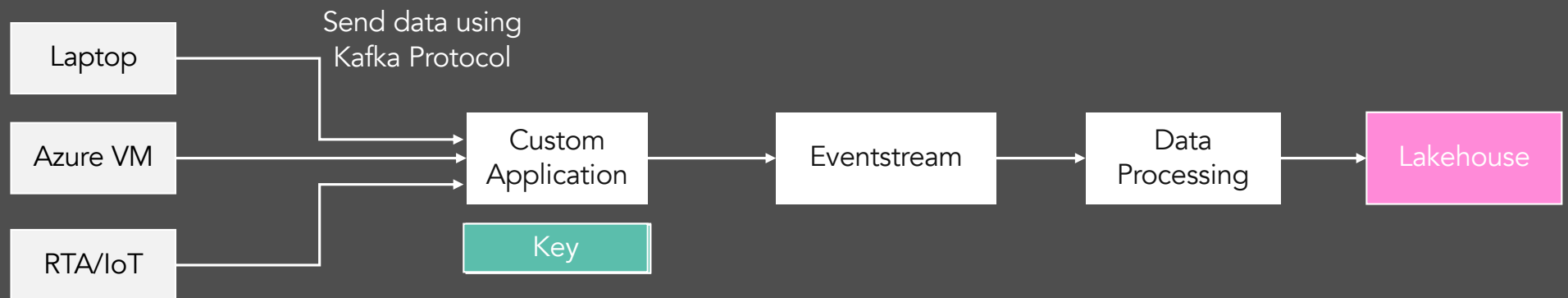
asyncio.run(send_data_to_eventstream())
```

Hands-On 1 : Architecture



Hands-On 2 : Kafka Protocol

- Lakehouse, custom application are supported as destinations only using without schema.



- Send data using Kafka Protocol.
- pip3 install kafka-python.

Hands-On 2 : Python Code Kafka

- Python code to send data to Eventstream using Kafka Protocol.

```
producer = KafkaProducer(  
    bootstrap_servers='<provided-by-es>',  
    security_protocol='SASL_SSL',  
    ssl_cafile='/etc/ssl/cert.pem', -- get this for your OS  
    sasl_mechanism='PLAIN',  
    sasl_plain_username='$ConnectionString',  
    sasl_plain_password='Endpoint...',  
    key_serializer=lambda x: json.dumps(x).encode('utf-8'),  
    value_serializer=lambda x: json.dumps(x).encode('utf-8'),  
)  
  
record = dict() -- should be of dictionary data type  
  
producer.send('<topic-name-provided-by-es>', value=record)
```

Spark Stream Processing



Search



Courses

Blog

About

Contact

FAQ

Login

(SPARK + KAFKA + More) Batch & Stream Processing - Data Engg. Vol 2

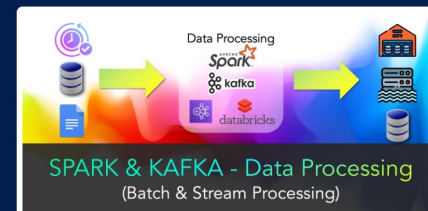


★★★★★ 5.0 (3 ratings)

Created by Soumyadeep Dey

🌐 English

Visit www.3aayaam.in



(SPARK + KAFKA + More) Batch & Stream Processing - Data Engg. Vol 2

Buy now for ₹4,999

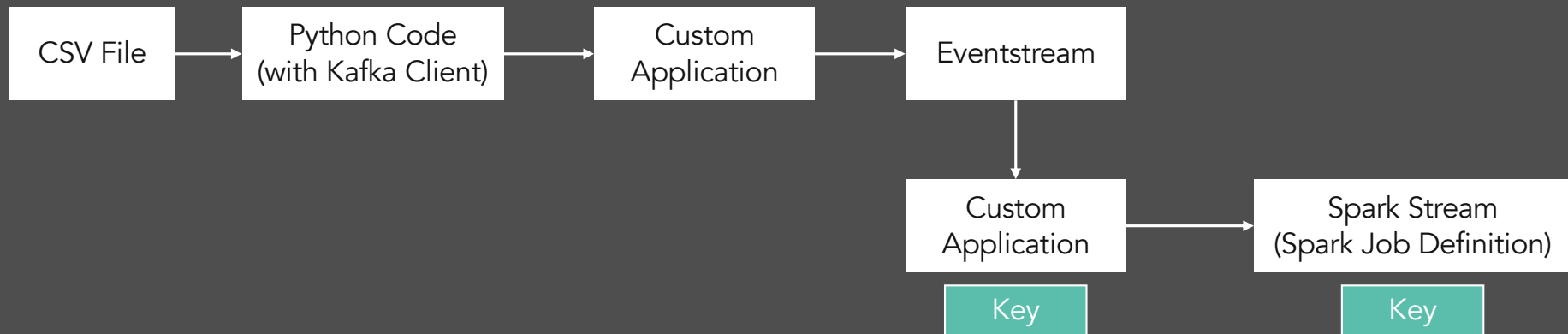
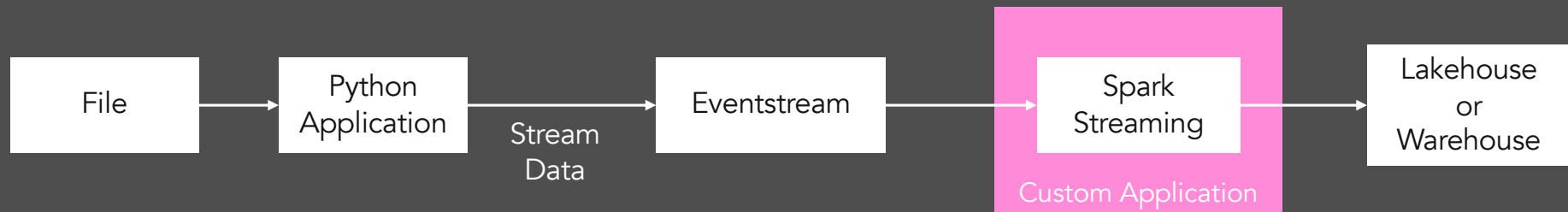
Language: English

Instructors: Soumyadeep Dey

Course Curriculum

36 Modules 📁 | 362 Sessions 📄 | 73 hour 21 min 51 sec Total Time ⌚

Hands-On 3 : Spark Streaming



Hands-On 3 : Python Code (Spark Streaming)

- PySpark code to receive data from Eventstream using Kafka Protocol.

```
df1 = spark.readStream.format('kafka') \
    .option('kafka.bootstrap.servers', '<bs-server-provided-by-eventstream>') \
    .option('subscribe', '<kafka-topic-provided-by-eventstream>') \
    .option("kafka.security.protocol", "SASL_SSL") \
    .option("kafka.sasl.mechanism", "PLAIN") \
    .option("kafka.sasl.jaas.config", <next-slide>) \
    .option("startingOffsets", "latest") \
    .load()

. . .

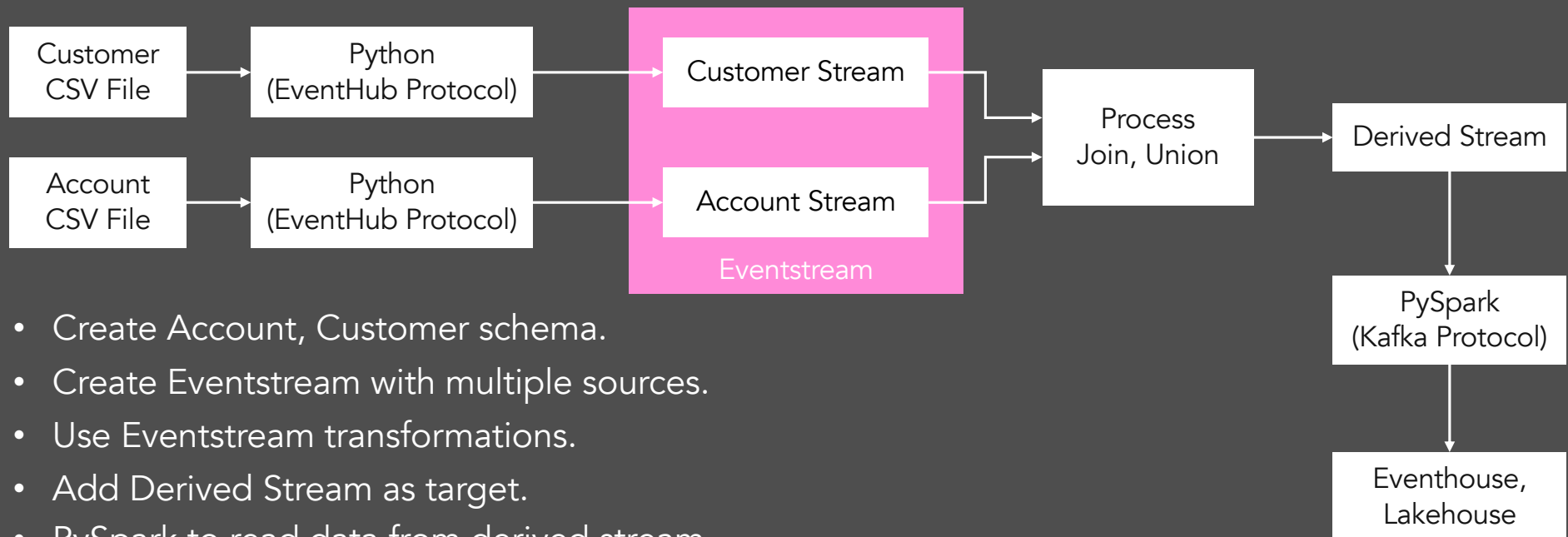
dfy = dfx.writeStream.format('delta').outputMode('append')\
    .option('checkpointLocation','Files/<folder>/<file-name>')\
    .trigger(processingTime='2 minute')\
    .toTable('<lakehouse-table>')
```

Hands-On 3 :

kafka.sasl.jaas.config

```
'org.apache.kafka.common.security.plain.PlainLoginModule  
required username="$ConnectionString"  
password="{CONNECTION_STRING}";'
```

Hands-On: Multiple Streams



- Create Account, Customer schema.
- Create Eventstream with multiple sources.
- Use Eventstream transformations.
- Add Derived Stream as target.
- PySpark to read data from derived stream.
- Write to Lakehouse.

EXERCISE - Eventstream

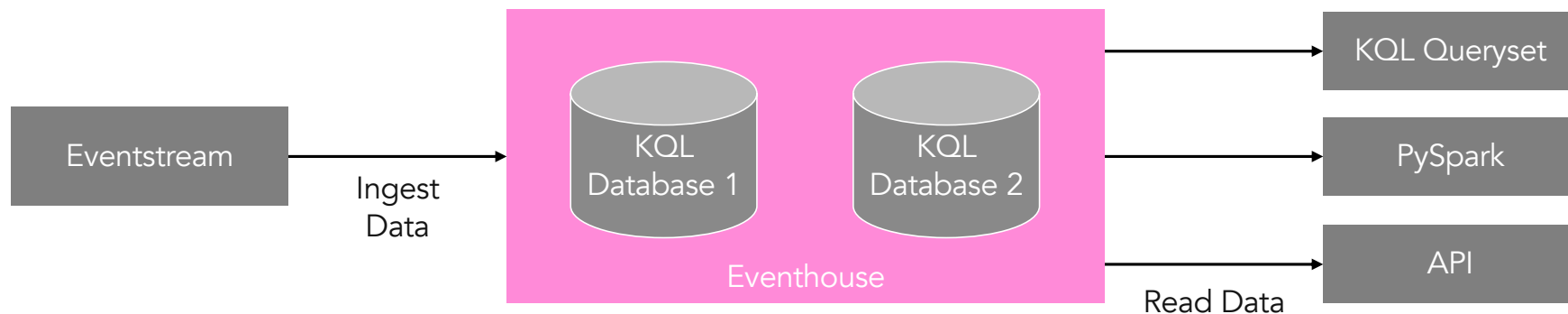
- Stream transaction data to Eventstream using Kafka protocol and store it in Lakehouse.
 - While sending data to Eventstream include an additional timestamp column using the code.
- Complete the following use cases. Each use case should have their individual Lakehouse table.
 - All complete transactions and their source, destination accounts and amount should be stored separately.
 - Find out the total amount transacted over the last 15 mins (hint: Spark event time processing. Event time should be the timestamp that was added by the code).
- Stream loan data in a separate Eventstream. Add a timestamp column.
 - Each loan type should have their separate tables.
 - At every 5 mins, find out the (total principal amount/1000) and store LH table.
 - At every 5 mins, find out the max term month and store in a LH table.

RTI Eventhouse

www.3aayaam.in

Eventhouse

- Real-time dashboard & analytics.
- Automatic scaling for storing large volume of data.
- Semi-structured, structured, time-series and text data can be stored.
- Eventhouse contains one or more Kusto (KQL) Database.



Kusto DB

- Can be used to stored Structured, Semi-structured and unstructured data.
- Object Hierarchy : Database → Table → Column.
- KQL (Kusto Query Language) is used to perform read operations on Kusto Databases. Writes are not possible.
- KQL Queryset – Used to run Kusto queries.

KQL Ingest Data into Tables

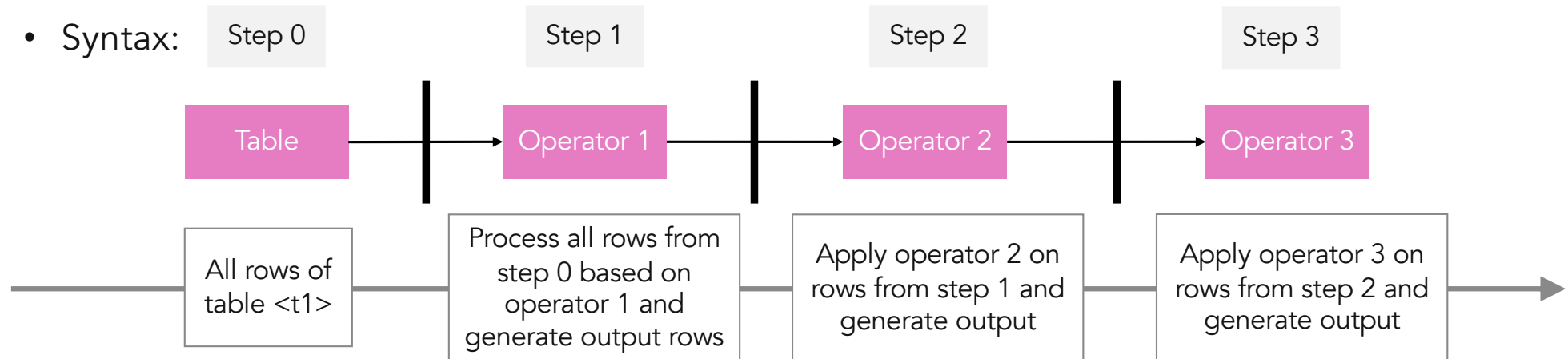
- Ingest data into tables from ADLS files (CSV, JSON, Parquet, Avro, ORC etc.)

```
.ingest async into table <tblname> ('abfss://<folder>;impersonate') with (format='csv')
```

- From Eventstream.

Kusto Query Language (KQL)

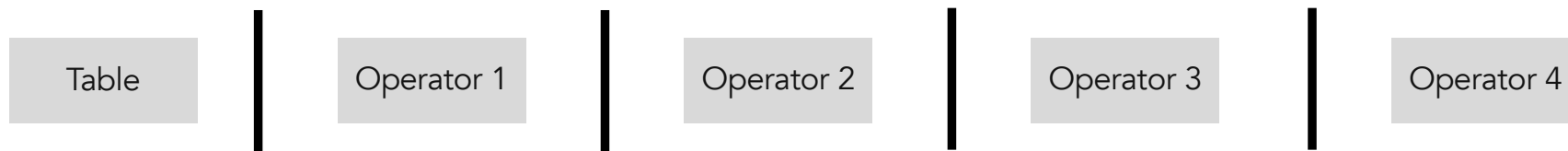
- Syntax:



- Case sensitive.
- Operators :

where, has, where contains, search, take, case, distinct, ago, bin, format_datetime, project, project-away, project-rename, extend, sort by, top, summarize, count, join, union, range, render, invoke

KQL Read



```
ledger | where ledger_type = "Equity" | where amount >= 10000 | take 10
```

```
ledger  
| where ledger_type = "Equity"  
| where amount >= 10000  
| take 10  
| extend perc = amount * 0.05
```

project

sort by

distinct

count

extend

top 10

KQL Aggregation

- max, min, avg, sum.
- count, count_distinct.

```
max(<col>)  
min(<col>)  
avg(<col>)  
sum(<col>)  
count()  
count_distinct(<col>)
```

- Window functions – for analytics.

```
row_number()  
row_rank_dense()  
prev()  
next()
```

- Time series analysis functions.

KQL – summarize, sort, range

- Summarize – Similar to GROUP BY in SQL.

```
summarize <col>=<aggregation> by <col1>, <col2>
```

- Sort – Order output result set by column values.

```
sort by <col1> asc, <col2> desc
```

- Range – Similar to BETWEEN in SQL.

```
range <col> from <value1> to <value2>
```

KQL Statements – let, declare

- Let Statement – Used to set, define and initialize variables.

```
let type="Equity";  
ledger | where ledger_type = type
```

- Query Parameter

```
declare query_parameters(<parm1>:<type1> = <value1>, <parm2>:<type2> = <value2>);  
ledger  
| where ledger_type = parm1  
| where amout = parm2
```

KQL Statements - set

- Set Statement – Set database configuration and properties.
 - servertimeout
 - request_user
 - request_readonly
 - request_readonly_hardline
 - request_block_row_level_security
 - queryconsistency
 - query_results_cache_max_age
 - query_language (kql, sql)
 - query_force_row_level_security

```
set Queryconsistency=strongconsistency;
```

KQL Render

- Generates visualizations of queries.
- Interactive dashboards.

```
kql query |  
render <chart> with (<property>=<value>)
```

- areachart – 1st col x-axis. Other numeric columns in y-axis.
- barchart – 1st col x-axis. Other columns in x-axis.
- columnchart – 1st col x-axis. Other numeric columns as y-axis.
- linechart – 1st col (should be numeric) x-axis. Other numeric cols in y-axis.

PropertyName	PropertyValue
accumulate	Whether the value of each measure gets added to all its predecessors. (true or false)
legend	Whether to display a legend or not (visible or hidden).
series	Comma-delimited list of columns whose combined per-record values define the series that record belongs to.
ymin	The minimum value to be displayed on Y-axis.
ymax	The maximum value to be displayed on Y-axis.
title	The title of the visualization (of type string).
xaxis	How to scale the x-axis (linear or log).
xcolumn	Which column in the result is used for the x-axis.
xtitle	The title of the x-axis (of type string).
yaxis	How to scale the y-axis (linear or log).
ycolumns	Comma-delimited list of columns that consist of the values provided per value of the x column.
ysplit	How to split the visualization into multiple y-axis values. For more information, see Multiple y-axes .
ytitle	The title of the y-axis (of type string).
anomalycolumns	Comma-delimited list of columns, which will be considered as anomaly series and displayed as points on the chart

KQL Render

- Generates visualizations of queries.
- Interactive dashboards.

```
kql query |  
render <chart> with (<property>=<value>)
```

- timechart – 1st col (datetime) x-axis.
Other numeric cols in y-axis.
- scatterchart – 1st col (numeric) x-axis.
Other numeric cols in y-axis.

PropertyName	PropertyValue
accumulate	Whether the value of each measure gets added to all its predecessors. (true or false)
legend	Whether to display a legend or not (visible or hidden).
series	Comma-delimited list of columns whose combined per-record values define the series that record belongs to.
ymin	The minimum value to be displayed on Y-axis.
ymax	The maximum value to be displayed on Y-axis.
title	The title of the visualization (of type string).
xaxis	How to scale the x-axis (linear or log).
xcolumn	Which column in the result is used for the x-axis.
xtitle	The title of the x-axis (of type string).
yaxis	How to scale the y-axis (linear or log).
ycolumns	Comma-delimited list of columns that consist of the values provided per value of the x column.
ysplit	How to split the visualization into multiple y-axis values. For more information, see Multiple y-axes .
ytitle	The title of the y-axis (of type string).
anomalycolumns	Comma-delimited list of columns, which will be considered as anomaly series and displayed as points on the chart

KQL Admin Commands

- Management Commands – Create table, drop table, modify table etc.
- Starts with a DOT (.).

```
.show databases
.show database datastats details identity policies
.show database <dbname> schema
.show tables | <operator> | ...
.create table <tblname> (<col>:<data-type>, <col2>:<data-type>)
.alter table <tblname> (<col>:<data-type>, <col2>:<data-type>)
.alter column <dbname>.<tblname>.<colname> type=<new-data-type>
.drop table <tblname>
.clear table <tblname> data
```

KQL Admin Commands

```
.create external table <tbl-name>  
dataformat=csv|parquet  
(h@'abfss://<uri>')  
  
.create materialized-view <mvname> on table <tbl-name> <KQL query>
```

KQL Join Statement

```
<table1(LeftTable)>
```

```
|
```

```
join kind=<join-type> <table2(RightTable)> on <join-conds>
```

Join Types:

- inner
- leftouter
- rightouter
- leftsemi
- rightsemi
- leftanti

```
$left.<t1-col1> == $right.<t2-col1>  
and  
$left.<t1-col2> == $right.<t2-col2>  
and  
$left.<t1-col3> == $right.<t2-col3>
```

```
union, lookup, find, parse, print, search, scan
```

KQL Table Policies

- Auto delete – Table is deleted automatically at expiry date.

```
.alter table <tblname> policy auto_delete @{ "ExpiryDate" : "2027-12-31" }
```

- Caching – Hot (local SSD+RAM) and Cold (Object Storage) caching.
 - Can be applied at database, table and mv level. Query results are cached automatically
- Scoping Queries – Only query data from hot cache. Improves query performance

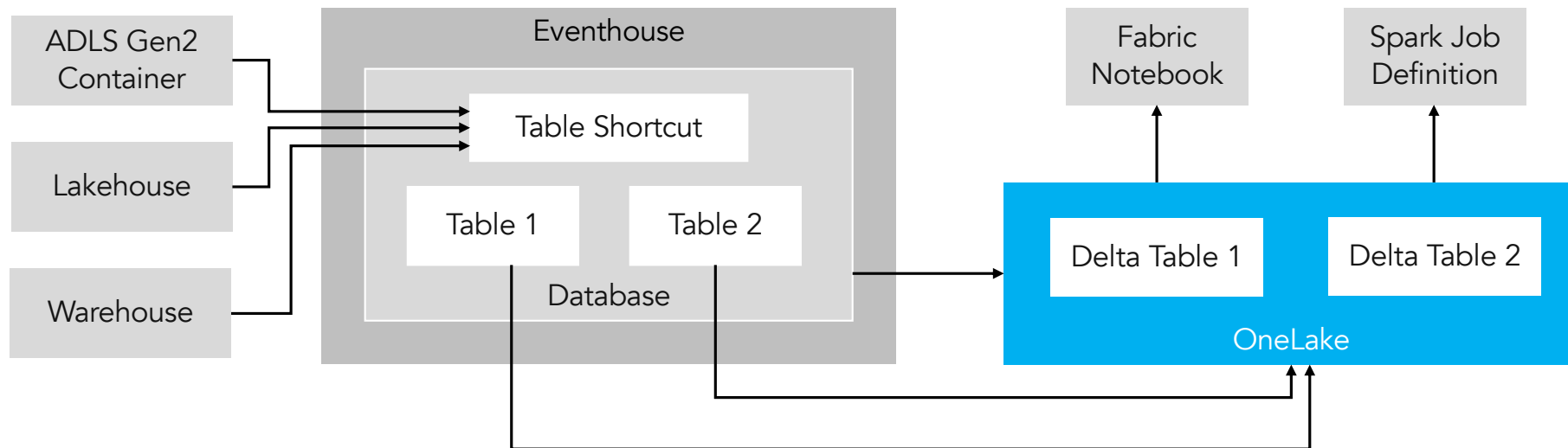
```
.alter database <dbname> policy caching hot = 120d  
.alter table <dbname>.<tblname> policy caching hot = 120d  
.clear table <tblname> cache streamingingestion schema  
.clear database cache streamingingestion schema  
.clear database cache query_results  
set query_datascope="hotcache"
```

KQL Table Policies

- Ingestion batching policy. Groups ingested data in small batches. Reduces resource utilization, provides better performance.
- IngestionTime policy. Adds \$IngestionTime column with data type datetime.
- Retention Policy. Used to automatically remove data from tables/MVs.
- Row Level Security (RLS) policy.

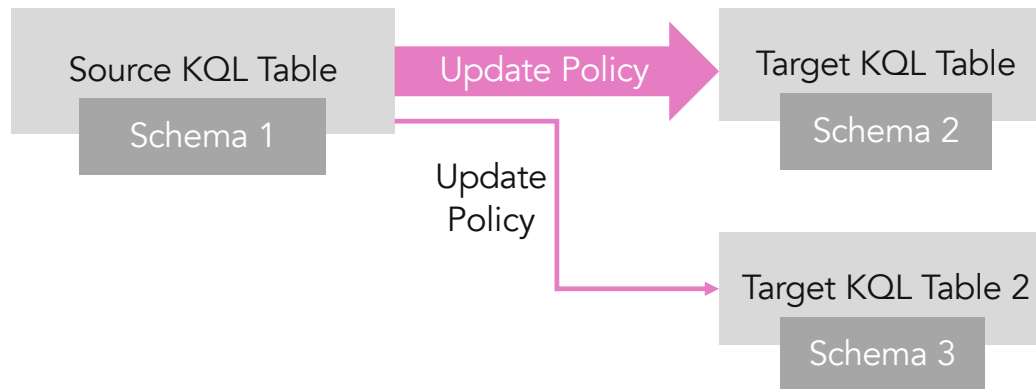
KQL Table Shortcut

- Eventhouse databases & tables can be exposed as Delta tables using OneLake.
- Delta tables can be queried using Spark, SQL etc.
- Shortcuts can be used for ADLS container, Lakehouse & Warehouse.



KQL Table Update Policies

- Similar to RDBMS Triggers.
- Update Policy – New data added to the source table can be appended to target table after transforming using KQL query.
- Multiple Update Policies can be defined on a single table.
- Update policy is defined on the target table.



```
.alter table <db>.<tgt_tbl> policy update  
  
[  
  {  
    "IsEnabled" : true,  
    "Source" : "<src_tbl>",  
    "Query" : "<kql_query>",  
    "IsTransactional" : true|false  
  }  
]
```

KQL Materialized View

- Similar to Materialized View in SQL.
- Complex aggregation query on a source table or another materialized view.
- Types:
 - Empty – Does not contain any data after creation. New data added to source table are added thereafter. Faster creation time.
 - Non-empty – MV gets created based on the existing records in source table. Newly added data gets appended thereafter. Longer creation time.
- Data deletion does not modify data in the MV.

```
.create materialized-view <mvname> on table <tbl_name> <KQL query>
```

KQL Functions

- Contains block of queries that can be reused.
- 2 types:
 - Stored functions – Function definition is stored as an object in the database. Can be called from other queries, multiple times.
 - Query-defined functions – Defined and used within a single query and not stored anywhere. Cannot be used once the query execution is complete. Defined using “let” statement.

```
.create-or-alter function (parm1:data-type1, parm2:data-type2)
{
    <let statement>
    <kql query>
}
```

KQL with Spark

Read:

```
accessToken = mssparkutils.credentials.getToken(kustoUri)

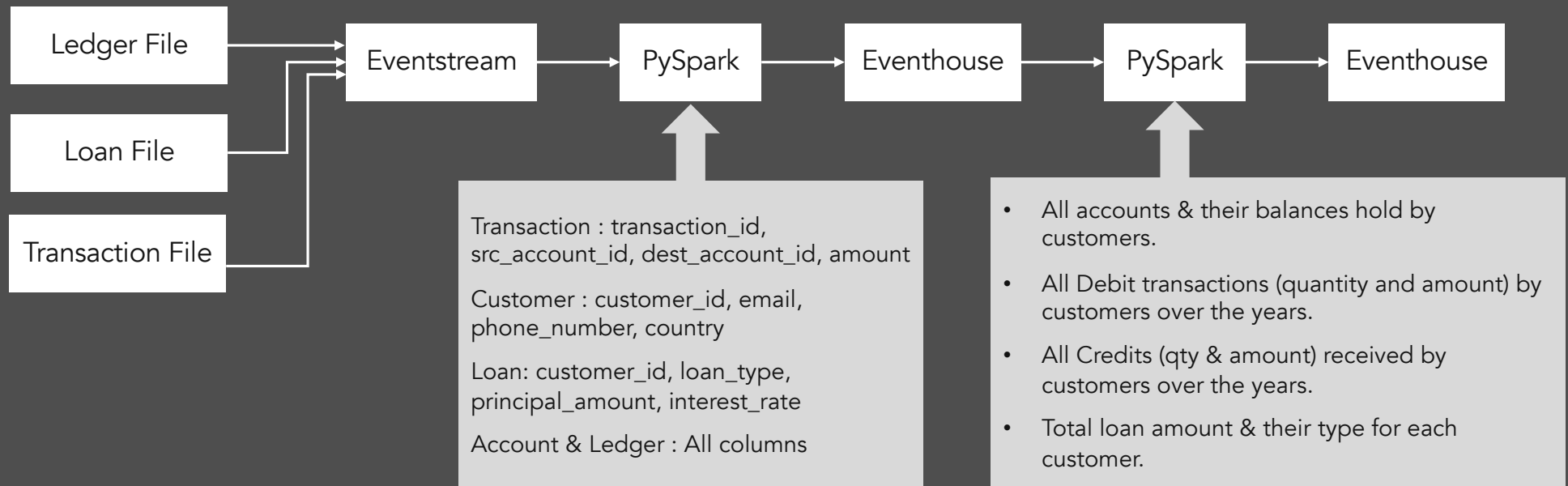
kustoDf = spark.read\
    .format("com.microsoft.kusto.spark.synapse.datasource")\
    .option("accessToken", accessToken)\
    .option("kustoCluster", kustoUri)\
    .option("kustoDatabase", database)\
    .option("kustoQuery", kustoQuery).load()
```

Write:

```
df_w.write.format("com.microsoft.kusto.spark.synapse.datasource")\
    .option("kustoCluster", kustoUri)\
    .option("kustoDatabase", database)\
    .option("kustoTable", table)\
    .option("accessToken", accessToken)\
    .option("tableCreateOptions", "CreateIfNotExist")\
    .mode("Append").save()
```

EXERCISE – Eventhouse

- Send the following datasets as streams to Eventstream and save them in Eventhouse after processing – Transaction, Ledger and Loan.
- Read from Eventhouse Kusto DB, process data and save them back to another Kusto DB.



EXERCISE – KQL

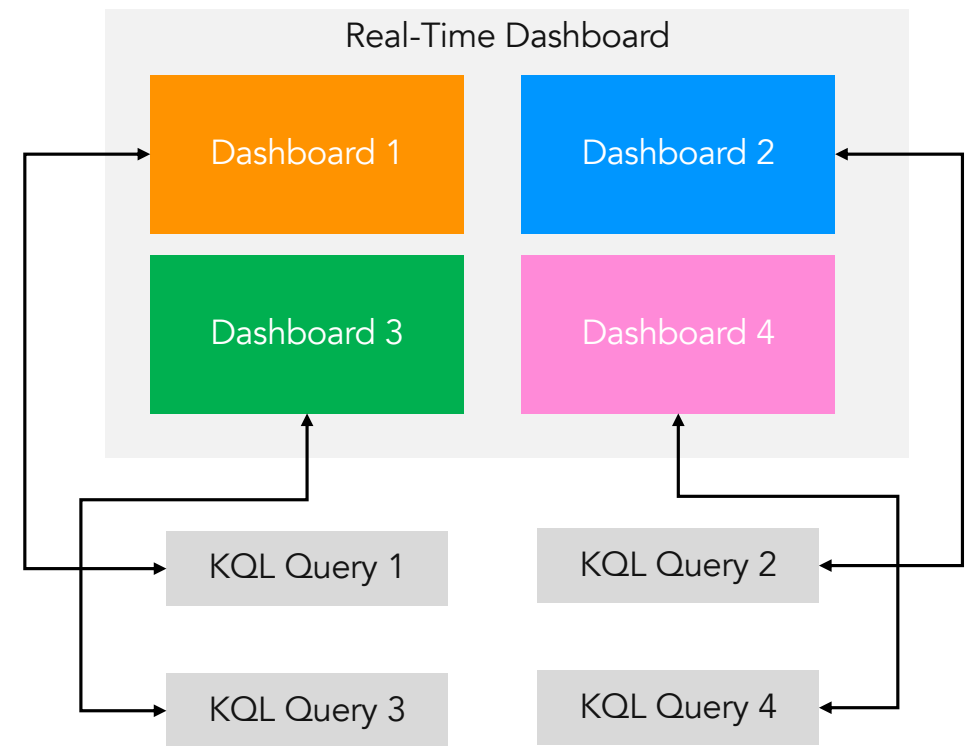
- Load all tables from CSV to Eventhouse. Solve the following business questions using KQL queries.
 - Account details of customers - customer details + account details of each customer.
 - Total no. of customers per country and their total balance (divide by 1000).
 - Total no. of accounts per customer per country.
 - Total no. of loans and loan amount per loan type and country.
 - All cross-country transactions and total amount per source country.
 - Total amount per year, per ledger type, per country.
 - Total interest payable for all loans, country-wise.
 - Details of completed transactions. Customer and their account details.
 - Latest loan for each country.
 - Total ledger entry for the year 2022 for each country and total transaction for the same duration.

RTI Real-Time Dashboard Activator

www.3aayaam.in

Real-Time Dashboard (RTD)

- Live dashboard for monitoring, visualization, operational analytics etc.
- Continuous data updates in dashboards with a min interval of 10 seconds.
- No-code experience.
- Git Integration.
- Data Sources – Eventhouse, Azure Data Explorer, Azure Monitor (Application Insights & Log Analytics).
- Alerts.

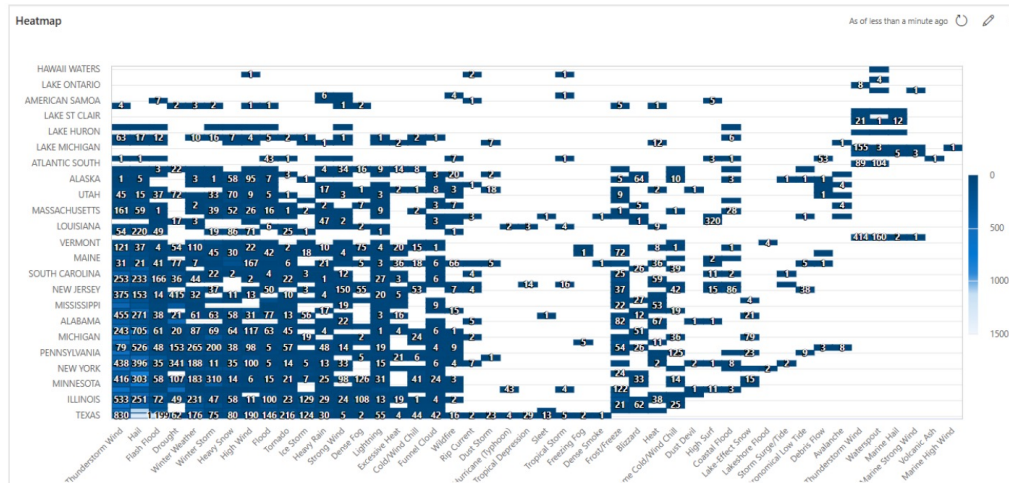


RTD Visualizations

- Funnel Chart.
- Heatmap

StormEvents

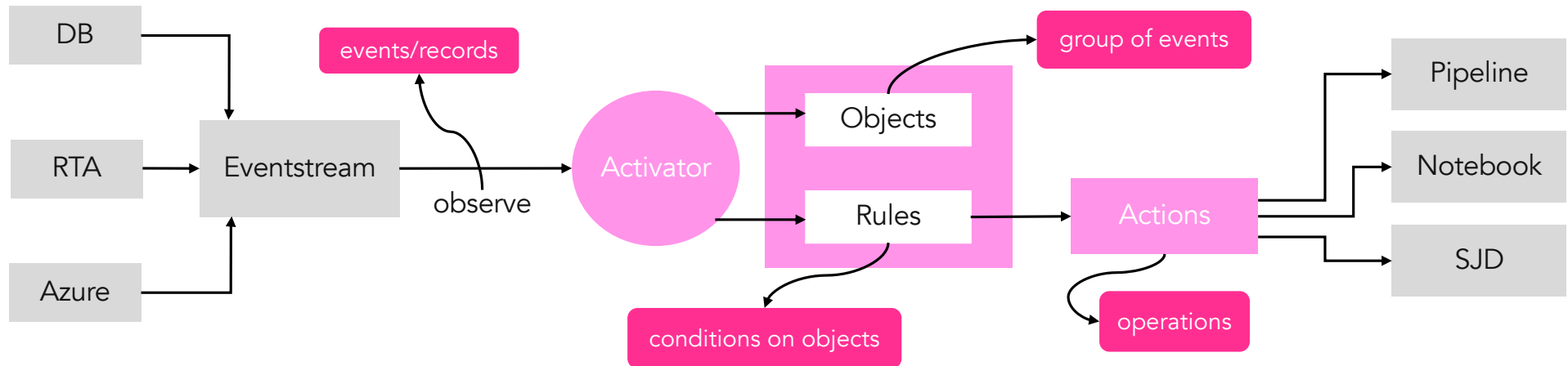
```
| summarize count(EventId) by State, EventType
```



Activator

- Trigger. No-code & Low latency.
- Monitors data sources and triggers downstream actions.
- Source: Eventstream, Real-Time Dashboard, Fabric Events, Azure Events, Power BI.
- Trigger Actions:
 - Pipeline.
 - Fabric Notebooks.
 - Spark Job Definitions.
 - Send message via Outlook, Teams etc.

Activator Architecture



- Activator observes events or records as objects.
- Checks defined rules or conditions, whether threshold has been broken or not.
- If broken, takes actions or performs operations.

Summary

www.3aayaam.in

- Real-Time Intelligence Solutions
- Eventstream
- Sources, Destinations, Processing, Schema
- Eventstream with EventHub & Kafka Protocol
- PySpark Streaming with Eventstream
- Eventhouse – Kusto DB
- Kusto Query Language (KQL)
- Read, Aggregation, Render, Admin, Join
- Table Shortcuts
- KQL DB with Spark
- Real-Time Dashboard
- Activator